



**Universidad**  
**Europea** VALENCIA

# Memoria Técnica

## Sistema de Gestión de Biblioteca



# **Índice**

## **1. Introducción**

## **2. Objetivos del Proyecto**

## **3. Tecnologías Utilizadas**

## **4. Estructura del Proyecto**

## **5. Diseño Orientado a Objetos**

### **5.1 Clase Material**

### **5.2 Herencia de Material**

### **5.3 Clase Usuario**

### **5.4 Tipos de Usuario**

### **5.5 Clase Préstamo**

### **5.6 Clase Reserva**

## **6. Aplicación de POO**

## **7. Lógica de Negocio**

**8. Persistencia de Datos**

**9. Servicios del Sistema**

**10. Interfaz de Usuario**

**11. Control de Errores**

**12. Ventajas del Diseño**

**13. Posibles Mejoras Futuras**

**14. Conclusión**

## **1. Introducción**

Este proyecto consiste en el desarrollo de un sistema de gestión de biblioteca utilizando Programación Orientada a Objetos (POO) en Python. El objetivo principal es permitir la administración de materiales bibliográficos, usuarios, préstamos y reservas mediante una aplicación organizada en módulos bien diferenciados.

El sistema implementa funcionalidades reales de una biblioteca: control de disponibilidad, préstamos con fechas límite, reservas con caducidad, sanciones a usuarios y almacenamiento persistente de datos mediante archivos JSON. La interfaz de usuario se ha desarrollado con tkinter, ofreciendo una experiencia visual más accesible que una simple consola.

## **2. Objetivos del Proyecto**

Los objetivos principales del proyecto son:

- Gestionar los materiales de una biblioteca (libros, revistas y recursos digitales).
- Registrar y administrar usuarios con distintos roles.
- Realizar préstamos y devoluciones con control de fechas y multas.
- Gestionar reservas de materiales no disponibles.
- Aplicar reglas de negocio como sanciones y límites de préstamos.
- Guardar y recuperar información mediante persistencia en archivos JSON.
- Aplicar correctamente los principios de Programación Orientada a Objetos.

## **3. Tecnologías Utilizadas**

- **Lenguaje de programación:** Python
- **Interfaz gráfica:** tkinter (biblioteca estándar de Python)
- **Persistencia de datos:** Archivos JSON
- **Paradigma:** Programación Orientada a Objetos (POO)
- **Entorno:** Aplicación de escritorio

## **4. Estructura del Proyecto**

El proyecto está estructurado de forma modular, separando responsabilidades y facilitando tanto el mantenimiento como el crecimiento futuro del sistema.

Biblioteca/

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| — main.py         | # Punto de entrada. Lanza la GUI con tkinter.       |
| — console_main.py | # Versión alternativa con interfaz por consola.     |
| — models/         | # Clases que representan las entidades del sistema. |
| — services/       | # Lógica de negocio y operaciones principales.      |
| — persistence/    | # Carga y guardado de datos en JSON.                |
| — ui/             | # Interfaz gráfica desarrollada con tkinter.        |
| — utils/          | # Utilidades y funciones auxiliares.                |
| — data/           | # Archivos JSON con los datos del sistema.          |

El archivo `main.py` actúa como punto de entrada principal. Inicializa el servicio central (`BibliotecaService`), que carga los datos desde JSON automáticamente, y lanza la interfaz gráfica (`BibliotecaApp`) sobre una ventana de tkinter.

Adicionalmente, `console_main.py` ofrece una interfaz alternativa por consola con menús interactivos, útil para pruebas o para entornos sin soporte gráfico.

La carpeta `models` contiene todas las clases que representan las entidades del dominio. La carpeta `services` agrupa la lógica de negocio. La carpeta `persistence` gestiona la lectura y escritura de archivos JSON. La carpeta `ui` contiene los componentes visuales de tkinter. La carpeta `utils` agrupa funciones auxiliares reutilizables. Finalmente, la carpeta `data` almacena los archivos JSON generados durante la ejecución.

## **5. Diseño Orientado a Objetos**

### **5.1 Clase Material**

La clase `Material` es la clase base del sistema para representar cualquier elemento prestable de la biblioteca.

**Atributos principales:**

- `_id`
- `_titulo`
- `_autor`
- `_categoria`
- `_disponible`
- `_veces_prestado`

**Métodos principales:**

- `prestar()`
- `devolver()`
- `esta_disponible()`
- `descripcion_corta()`
- `to_dict()`

Esta clase aplica encapsulación mediante atributos protegidos (prefijo `_`) y sirve como base para la reutilización mediante herencia.

### **5.2 Herencia de Material**

El sistema utiliza herencia para especializar distintos tipos de materiales:

**Clase Libro** — hereda de `Material` y añade:

- `_isbn`

**Clase Revista** — hereda de `Material` y añade:

- `_numero_edicion`

**Clase RecursoDigital** — hereda de `Material` y añade:

- `_url`
- `_formato`

Gracias a la herencia, todas las clases comparten un comportamiento común y pueden tratarse de forma polimórfica en el resto del sistema.

### 5.3 Clase Usuario

Representa a los usuarios registrados en la biblioteca.

#### **Atributos principales:**

- `_id`
- `_nombre`
- `_sancionado`
- `_prestamos_activos`
- `_max_prestamos`

#### **Métodos principales:**

- `esta_sancionado()`
- `puede_prestar()`
- `sancionar()`
- `levantar_sancion()`
- `descripcion_corta()`

### 5.4 Tipos de Usuario

El sistema incluye distintos perfiles de usuario mediante herencia:

- **Socio** — usuario estándar con límite de préstamos definido.
- **Bibliotecario** — puede gestionar materiales y usuarios.
- **Administrador** — acceso completo al sistema.

Cada tipo puede tener comportamientos o límites diferentes según su rol.

### **5.5 Clase Préstamo**

La clase `Prestamo` representa la relación entre un usuario y un material.

#### **Funcionalidades:**

- Registrar fecha de préstamo.
- Calcular fecha de vencimiento.
- Registrar devoluciones.
- Detectar préstamos vencidos.
- Calcular multas por retraso.

#### **Métodos principales:**

- `devolver()`
- `esta_vencido()`
- `calcular_multa()`
- `descripcion_corta()`

### **5.6 Clase Reserva**

La clase `Reserva` permite reservar materiales que no están disponibles en ese momento.

#### **Funcionalidades:**

- Registrar reservas con fecha de caducidad.
- Cancelar reservas.
- Verificar si una reserva sigue activa.

#### **Métodos principales:**

- `esta_caducada()`
- `cancelar()`
- `esta_activa()`

## **6. Aplicación de POO**

**Encapsulación** Los atributos se definen con guion bajo (\_) para evitar el acceso directo desde el exterior y forzar el uso de métodos controlados. Ejemplo: `self._titulo`, `self._disponible`.

**Herencia** Se utiliza herencia cuando existe una relación real "es-un" entre clases. Ejemplo: `class Libro(Material)`. Esto evita duplicación de código y clarifica el modelo de dominio.

**Polimorfismo** Todos los materiales comparten métodos comunes como `descripcion_corta()` y `to_dict()`. Esto permite tratar libros, revistas y recursos digitales de manera uniforme en colecciones y servicios, sin necesidad de distinguir el tipo concreto en cada operación.

**Relaciones entre objetos** El sistema establece relaciones de composición entre entidades:

- Un `Prestamo` contiene un `Usuario` y un `Material`.
- Una `Reserva` contiene un `Usuario` y un `Material`.

## **7. Lógica de Negocio**

El sistema implementa reglas reales de funcionamiento de una biblioteca:

- No se puede prestar un material que no esté disponible.
- Un usuario sancionado no puede realizar préstamos.
- Existe un límite máximo de préstamos activos por usuario.
- Los préstamos tienen fecha de vencimiento automática.
- Se calculan multas en función de los días de retraso.
- Las reservas tienen fecha de caducidad y se gestionan en orden.

Estas reglas garantizan un comportamiento coherente y realista del sistema, y están implementadas en la capa de servicios para mantener los modelos limpios.

## **8. Persistencia de Datos**

La aplicación guarda y recupera la información utilizando archivos JSON almacenados en la carpeta `data/`:

- `materiales.json`
- `usuarios.json`
- `prestamos.json`
- `reservas.json`

El servicio principal (`BibliotecaService`) carga automáticamente estos archivos al arrancar la aplicación y guarda los cambios tras cada operación relevante.

Cada clase del modelo implementa un método `to_dict()` que convierte el objeto en un diccionario serializable, y un método de clase (o función equivalente en persistencia) que reconstruye el objeto desde ese diccionario.

### **Ventajas del uso de JSON:**

- Fácil lectura e inspección manual.
- Compatible de forma nativa con Python mediante el módulo `json`.
- Persistencia sencilla sin necesidad de base de datos externa.

## **9. Servicios del Sistema**

La carpeta `services` contiene la lógica principal de la aplicación, separada del modelo:

- `biblioteca_service.py` — gestiona operaciones generales sobre materiales y usuarios, y actúa como coordinador central del sistema.
- `prestamo_service.py` — gestiona la creación de préstamos, devoluciones, detección de vencimientos y cálculo de multas.
- `reserva_service.py` — gestiona la creación, activación, caducidad y cancelación de reservas.

Esta separación permite que las clases del modelo se mantengan centradas en representar entidades, mientras que las reglas de negocio se implementan en un lugar independiente y fácil de modificar.

## **10. Interfaz de Usuario**

El sistema ofrece dos interfaces:

**Interfaz gráfica (principal):** desarrollada con tkinter, accesible desde `main.py`. Permite interactuar con el sistema de forma visual mediante ventanas, formularios y botones. La clase `BibliotecaApp` (en `ui/gui.py`) recibe el servicio central como dependencia y construye la interfaz sobre él.

**Interfaz por consola (alternativa):** accesible desde `console_main.py`, con menús interactivos numerados. Es útil para pruebas rápidas o para entornos donde no esté disponible una pantalla gráfica.

Funcionalidades disponibles desde ambas interfaces:

- Ver y gestionar materiales
- Registrar y modificar usuarios
- Realizar préstamos y devoluciones
- Crear y consultar reservas
- Consultar informes del sistema

## **11. Control de Errores**

El sistema incorpora validaciones en la capa de servicios para evitar estados incoherentes:

- Verificación de disponibilidad antes de cualquier préstamo.
- Comprobación de sanciones activas sobre el usuario.
- Control del límite máximo de préstamos por usuario.
- Validación del estado de reservas antes de activarlas.
- Mensajes de error claros que se muestran al usuario en la interfaz.

El objetivo es que el programa nunca explote de forma inesperada y que cualquier operación inválida se comunique de manera comprensible.

## **12. Ventajas del Diseño**

El diseño modular del proyecto aporta las siguientes ventajas:

- Código más organizado y legible gracias a la separación por responsabilidades.
- Mayor reutilización de código mediante herencia y polimorfismo.
- Fácil mantenimiento: modificar una capa no obliga a tocar las demás.
- Escalabilidad: añadir nuevos tipos de materiales o usuarios requiere mínimos cambios.
- Testabilidad: los servicios pueden probarse de forma independiente al modelo y a la interfaz.

### **13. Posibles Mejoras Futuras**

- Migración a una base de datos relacional (SQLite o PostgreSQL).
- Sistema de login con autenticación de usuarios.
- Generación automática de informes en PDF o CSV.
- Notificaciones automáticas por vencimiento de préstamos.
- API REST para acceso remoto al sistema.
- Panel de administración web.
- Pruebas automatizadas con pytest.

### **14. Conclusión**

El proyecto desarrollado cumple los requisitos principales de un sistema de gestión de biblioteca utilizando Programación Orientada a Objetos. Se han aplicado correctamente los conceptos fundamentales de encapsulación, herencia, polimorfismo y separación de responsabilidades.

El sistema cuenta con dos modos de uso (interfaz gráfica con tkinter e interfaz por consola), persistencia de datos en JSON, lógica de negocio real implementada en servicios independientes y una estructura modular que facilita su ampliación futura.

El resultado es una aplicación funcional, bien organizada y coherente con las exigencias técnicas del reto planteado por DynamizaTIC.